

RASPBERRY PI

Personal Home NAS Server

From Tiny Pi to Powerful Storage: A Complete DIY Build Guide

Prepared by:

Bimsara Janakantha

Computer Engineer Undergraduate

University of Peradeniya

Sri Lanka

Project Version: 1.0

Date: July 2025

Status: Completed

1. Project Overview

This document is a full record of the design, build, and configuration of a personal Network Attached Storage (NAS) server built around a Raspberry Pi 5. The goal of the project was to create a reliable, self-hosted home storage system that works similarly to commercial cloud services like OneDrive, but runs entirely on private hardware with no monthly subscription fees.

The system is built inside a recycled desktop PC case and uses enterprise-grade hardware to achieve roughly 8 TB of protected RAID 5 storage. It supports multiple users with isolated storage quotas, is accessible over the internet through a secure tunnel, and provides a OneDrive-style sync experience through Nextcloud.

The project was completed in several phases, covering hardware assembly, operating system setup, RAID configuration, user management, network sharing, internet access via Cloudflare, and final client setup for web, mobile, and desktop users.

2. Hardware Configuration

2.1 Bill of Materials

The following components were used in the build:

Component	Qty	Notes
Raspberry Pi 5 (8GB)	1	Main computer board
MicroSD Card (64 GB)	1	OS boot drive
Geekworm Penta SATA Hat	1	Supports up to 5 SATA drives via PCIe
PCIe FPC Ribbon Cable	1	Connects Pi 5 to Penta SATA Hat
ATX Desktop Power Supply (650W)	1	Powers drives and the Pi
Old Desktop PC Case	1	Enclosure with existing fans and LEDs
Industrial SAS HDDs (4 TB each)	3	Enterprise-grade drives
SATA HDD (1 TB)	1	Standard desktop drive for web server
SAS-to-SATA Adapters	3	One per SAS drive
SATA Data Cables	4	One per drive
SATA Power Cables / Splitters	As needed	From ATX PSU
Latching Push Button	1	Replaces a momentary power switch
Male DC Barrel Jack (5.5mm/2.1mm)	1	For Pi power input via SATA Hat
Terminal Block Connectors	Several	For safe wire terminations
Active 5V Buzzer	1	Temperature warning alert
Green LED (Power Indicator)	1	Already in the PC case
Red LED (Disk Activity)	1	Already in the PC case

Component	Qty	Notes
PC Case Cooling Fans (12V)	2	Additional ventilation
Shrink Tubing	Assorted	For connection insulation
Zip Ties Pack	1	Cable management
Soldering Iron and Solder	1 set	For wiring connections
Multimeter	1	For continuity and voltage checks

2.2 Phase 1 — Component Verification

Before starting the assembly, every component was individually tested. This approach avoids the frustration of diagnosing failures inside a fully built system.

1. Raspberry Pi 5 — Connected to a monitor via micro-HDMI with a USB keyboard and SD card to confirm it boots correctly.
2. MicroSD Card — Raspberry Pi Imager was used on a Windows PC to write Raspberry Pi OS to the card and verify the write completed without errors.
3. Penta SATA Hat — Physically inspected for damage before assembly. Full functionality was verified after the complete build.
4. ATX Power Supply — Performed a paperclip test by shorting the PS_ON pin (Pin 16, green wire) to a GND pin (black wire) on the 24-pin ATX connector. The PSU fan spun and the multimeter confirmed approximately 12V on the yellow wire. The jumper was removed before continuing.
5. SAS Drives — Each drive was connected individually to a PC with an HBA card to confirm it spun up and was detected.
6. SATA HDD (1 TB) — Connected to a PC SATA port and confirmed detection using Disk Management.
7. SAS-to-SATA Adapters — Visually inspected for bent pins and confirmed each adapter was seated firmly.
8. PC Case — Checked the power button, LEDs, and existing fans for physical integrity.

2.3 Phase 2 — Drive Preparation

All drives were wiped and initialized before being placed in the enclosure. This was done by connecting each drive to a Windows laptop using a USB-to-SATA adapter, then using the built-in diskpart utility.

Steps performed for each drive:

9. Opened Command Prompt as Administrator and launched diskpart.
10. Listed all available disks using the list disk command and identified the correct disk number.
11. Selected the target disk using select disk X.
12. Wiped the disk completely using the clean command.
13. Initialised the disk with a GPT partition table using convert gpt (required for drives larger than 2 TB).

Warning: The clean command permanently erases all data on the selected disk. The correct disk number was triple-checked using the list disk and comparing sizes before proceeding.

2.4 Phase 3 — Mechanical Assembly

With drives tested and prepared, all components were mounted securely inside the PC case.

- Side panels were removed, and the 3.5-inch drive bays were identified.
- Each HDD was mounted using M3 or UNC 6-32 screws, with at least one bay gap between drives to allow airflow.
- The ATX PSU was secured into the rear PSU bracket with four retaining screws.
- The Raspberry Pi was mounted on standoffs with good airflow, away from the PSU fan exhaust.
- Cables were routed loosely at this stage and finalized during the cable management phase.

2.5 Phase 4 — Drive Connectivity

The SAS drives were connected via adapters, and all drives were wired to the Penta SATA Hat.

14. SAS-to-SATA adapters were pressed firmly onto each of the three SAS drives until fully seated.
15. SATA data cables were connected from each drive (including the 1 TB SATA HDD) to the corresponding ports on the Penta SATA Hat. Each port was labelled for reference during software configuration.
16. SATA power cables from the ATX PSU were connected to each drive. A SATA power splitter was used since the PSU did not have enough native SATA power connectors.
17. Each connection was given a firm tug to verify it was locked in place.

2.6 Phase 5 — ATX Power Supply Integration

The ATX PSU requires a permanent on/off mechanism since it is no longer controlled by a motherboard. The PC case's existing power button was rewired to control the PSU directly.

5a — Replace the Momentary Switch with a Latching Push Button

Standard PC power buttons are momentary (press and release = single pulse). To keep the ATX PSU powered on, a latching button is needed (press once to switch on, press again to switch off).

18. Locate the power switch connector on the front panel cable bundle.
19. Desoldered the existing momentary push button from the case's power switch header.
20. Soldered in a new latching push button, using shrink tubing over each solder joint.
21. Routed the two wires to a terminal block for a clean connection to the ATX connector.

5b — Wire PS_ON to the Latching Button

- 22. Identified the PS_ON pin (Pin 16, green wire) and a GND pin on the ATX 24-pin connector.
- 23. Tapped these two wires near the connector end (not near the PSU).
- 24. Connected the PS_ON and GND wires to the two terminals of the latching push button via terminal blocks.

Note: When the button is latched closed, it shorts PS_ON to GND, which signals the ATX PSU to power on. This mirrors what a motherboard does when the power button is pressed. The ATX PSU stays in standby until its PS_ON pin is pulled below approximately 0.8V.

2.7 Phase 6 — GPIO Wiring (LEDs and Buzzer)

The PC case's built-in LEDs and an active buzzer were wired to the Raspberry Pi GPIO to provide real-time visual and audio feedback.

Warning: Raspberry Pi GPIO pins operate at 3.3V with a maximum current of approximately 16mA per pin. A current-limiting resistor (330 ohm) was used in series with each LED to prevent GPIO damage.

Function	GPIO (BCM)	Physical Pin	Notes
Power LED (Green)	GPIO 17	Pin 11	330 ohm resistor in series
Disk Activity LED (Red)	GPIO 27	Pin 13	330 ohm resistor in series
Temperature Buzzer	GPIO 16	Pin 36	Via NPN transistor — buzzer needs 5V
GND (shared)	—	Pin 6, 9, 14...	Multiple GND pins available
5V (for buzzer circuit)	—	Pin 2 or 4	Do not draw more than 500mA total

The active buzzer requires 5V but GPIO pins only output 3.3V. An NPN transistor (2N2222) was used as a switch: GPIO 16 drives the transistor base through a 1k ohm resistor, the buzzer is connected between 5V and the collector, and the emitter goes to GND.

2.8 Phase 7 — Penta SATA Hat and PCIe Connection

- 25. The cooling fan supplied with the Penta SATA Hat was attached and oriented to draw air across the Hat's bridge chip.
- 26. The PCIe FPC ribbon cable was connected by opening the FPC latch on the Raspberry Pi 5's PCIe connector (J2), inserting the cable with contacts facing down, and closing the latch firmly. The other end was connected to the Hat in the same way.
- 27. The Penta SATA Hat was mounted on standoffs beside the Pi (not stacked on top) using the PCIe cable connection rather than the 40-pin GPIO header.
- 28. The FPC cable was checked to ensure no sharp bends or kinks.

2.9 Phase 8 — Raspberry Pi Power (12V to DC Jack)

The Penta SATA Hat includes a DC barrel jack input that accepts 12V and provides regulated power to both the Hat and the Raspberry Pi 5, removing the need for a separate USB-C power supply.

29. A spare SATA power connector from the ATX PSU was identified. Yellow wires carry +12V, and black wires are GND. Voltage was verified with a multimeter (~12V DC).
30. A 5.5mm/2.1mm male DC barrel jack was prepared: the yellow (+12V) wire was soldered to the centre pin (positive), and the black (GND) wire was soldered to the outer barrel (negative). Shrink tubing was applied to each joint.
31. The DC jack was plugged into the power input on the Penta SATA Hat.

Warning: Centre-positive polarity was confirmed before powering on. Reversed polarity would damage both the Hat and the Pi.

2.10 Phase 9 — Cooling and Cable Management

Cooling

Two additional 80mm fans were mounted in the case — one at the front as an intake and one at the rear as an exhaust. Both fans were connected to 12V (yellow) and GND (black) from the ATX PSU. The airflow path runs front to back: intake at the front draws air past the drives, and the exhaust at the rear expels hot air.

Cable Management

- Cables were grouped by type: power cables together, data cables together, and GPIO signal wires together.
- Zip ties were used at regular intervals (every 10 to 15 cm) to bundle cables neatly along the case frame, away from fan blades.
- Shrink tubing was applied to every exposed solder joint and any spliced wire.
- Service loops (a small amount of slack near connectors) were left to allow unplugging without straining the joint.

3. Architecture and Operating System Setup

3.1 OS Selection

Ubuntu Server 24.04 LTS (64-bit) was chosen as the base operating system because it is stable, fully supported on the Raspberry Pi 5, and pairs well with CasaOS. CasaOS is a lightweight web dashboard that installs on top of Ubuntu and provides a beginner-friendly visual interface with a built-in Docker-based App Store.

3.2 OS Installation

The installation was done headlessly — without ever plugging a monitor or keyboard into the Pi — using the official Raspberry Pi Imager tool.

32. Raspberry Pi Imager was downloaded and installed on a Windows PC.
33. The MicroSD card was inserted and Ubuntu Server 24.04 LTS (64-bit) was selected as the OS with Raspberry Pi 5 as the device.
34. Before flashing, OS Customisation settings were configured: hostname set to pinas, a username and password were created, Wi-Fi credentials were entered, and SSH was enabled with password authentication.
35. The SD card was flashed and verified, then inserted into the powered-down Raspberry Pi 5.
36. All four hard drives were connected, the power supply was switched on, and the Pi was powered up.
37. After 3 to 5 minutes, the Pi's IP address was found using the router's admin page and confirmed with the hostname pinas.

3.3 Health Monitoring Setup

A Python monitoring script was created to provide real-time hardware feedback via the GPIO-connected LEDs and buzzer.

Dependencies

```
sudo apt update
sudo apt install python3-gpiozero -y
```

Activity LED Overlay

The Pi's firmware was configured to map the internal activity LED signal to GPIO 27 by adding the following line to `/boot/firmware/config.txt`:

```
dtoverlay=act-led,gpio=27
```

Monitor Script

The health monitor script was created at `/usr/local/bin/heath/health_monitor.py`. It performs the following:

Indicator	Condition
Disk LED ON (GPIO 17)	Active disk read/write detected
Buzzer beeping (GPIO 16)	CPU temperature is 60°C or above
Log entry written	Every 5 seconds — records uptime and temperature

Disk activity is polled every 50ms for a responsive LED flicker effect. Temperature is checked every 5 seconds and logged to a file at `/usr/local/bin/heath/nas_status.log`.

Running as a System Service

The script was configured as a systemd service (`nashealth.service`) so it starts automatically on every boot:

```
sudo systemctl daemon-reload
sudo systemctl enable nashealth.service
sudo systemctl start nashealth.service
```

4. Storage Configuration

4.1 Drive Identification

Before formatting anything, the drive names were confirmed using the `lsblk` command to avoid accidentally wiping the boot drive. The three 4TB drives appeared as `sda`, `sdb`, and `sdc`, while the 1TB drive appeared as `sdd`. The boot SD card appeared as `mmcblk0`.

4.2 Installing the RAID Tool

`mdadm`, the standard Linux utility for managing software RAID arrays, was installed:

```
sudo apt update && sudo apt install mdadm -y
```

4.3 Building the RAID 5 Array

The 3 x 4TB drives were combined into a single protected virtual drive called `/dev/md0` using RAID 5

```
sudo mdadm --create --verbose /dev/md0 --level=5 --raid-devices=3 /dev/sda  
/dev/sdb /dev/sdc
```

RAID 5 was chosen because it provides a balance of storage capacity and fault tolerance: one drive can fail without any data loss. The total usable capacity with three 4TB drives in RAID 5 is approximately 8TB.

4.4 Formatting the Drives

Both storage volumes were formatted with the `ext4` filesystem:

```
sudo mkfs.ext4 /dev/md0          # 8TB RAID array  
sudo mkfs.ext4 /dev/sdd         # 1TB web server drive
```

4.5 CasaOS Installation

CasaOS was installed using the official one-line installer. It automatically sets up Docker and all necessary background services:

```
curl -fsSL https://get.casaos.io | sudo bash
```

Once installation completed, the CasaOS dashboard was accessed through the Pi's local IP address in a web browser. The RAID array and the 1TB drive were both visible in the Storage Manager, and storage volumes were created for each.

4.6 Debug — RAID Array Reboot Issues

After the first reboot, the RAID array appeared as `/dev/md127` instead of `/dev/md0`. This is a common Linux behaviour: the system scanned the drives during early boot and, finding no configuration file, automatically assigned a high number to avoid conflicts.

The permanent fix involved three steps:

38. Stopped the temporary array: `sudo mdadm --stop /dev/md127`
39. Manually reassembled it as md0: `sudo mdadm --assemble /dev/md0 /dev/sda /dev/sdb /dev/sdc`
40. Saved the configuration and updated the boot process:

```
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
sudo update-initramfs -u
```

After a reboot, the array correctly appeared as md0.

4.7 Making the Mount Permanent

The UUID of the RAID array was obtained using `sudo blkid /dev/md0`, and a permanent mount entry was added to `/etc/fstab`:

```
UUID=<your-uuid-here> /mnt/NAS_Storage ext4 defaults,nofail 0 0
```

Note: The *nofail* option was added so that if the RAID array is still spinning up during boot, Ubuntu continues booting rather than hanging.

CasaOS was then restarted to refresh its dashboard:

```
sudo systemctl restart casaos.service
```

4.8 RAID Build Verification

After the RAID sync completed, four commands were run to confirm the array was fully healthy:

41. `cat /proc/mdstat` — confirmed active raid5 with the status [3/3] [UUU], meaning all three drives are up and synced.
42. `sudo mdadm --detail /dev/md0` — confirmed RAID Level 5, Array Size approximately 7.28 TiB, and State as clean.
43. `df -h | grep md0` — confirmed Size of approximately 7.3T and Mount Point of `/mnt/NAS_Storage`.
44. `lsblk` — confirmed the three 4TB drives (sda, sdb, sdc) each branch to md0, which is mounted at `/mnt/NAS_Storage`.

The [3/3] [UUU] result confirmed that the RAID 5 array was fully synced, healthy, and ready.

5. User Management and Storage Quotas

5.1 Architecture Overview

The NAS was designed to serve two regular users alongside the admin account. Each user gets a private, isolated storage area with a hard size limit. The 1TB web server drive is restricted to the admin account only.

5.2 Lock Down the Web Server Drive

The 1TB drive was secured so only the admin account can access it:

```
sudo chown -R admin:admin /mnt/Web_Server
sudo chmod -R 700 /mnt/Web_Server
```

Note: *chmod 700 means the owner has full control and all other users have zero access.*

5.3 Create User Accounts

Two Ubuntu system accounts were created for the regular users (referred to here as Alice and Bob):

```
sudo adduser alice
sudo adduser bob
```

Both users were also added to the Samba password registry so they can access the NAS over the local network:

```
sudo smbpasswd -a alice
sudo smbpasswd -a bob
```

5.4 Create Private Storage Folders

Dedicated folders were created for each user on the 8TB RAID array and locked so they cannot access each other's data:

```
sudo mkdir /mnt/NAS_Storage/alice
sudo mkdir /mnt/NAS_Storage/bob
sudo chown alice:alice /mnt/NAS_Storage/alice
sudo chown bob:bob /mnt/NAS_Storage/bob
sudo chmod 700 /mnt/NAS_Storage/alice
sudo chmod 700 /mnt/NAS_Storage/bob
```

5.5 Enforce Storage Quotas

Linux disk quotas were enabled to hard-limit how much space each user can consume.

45. The `/etc/fstab` entry for the RAID array was updated to include quota tracking options:

```
UUID=<uuid> /mnt/NAS_Storage ext4 defaults,nofail,usrquota,grpquota 0 0
```

46. The ext4 filesystem was upgraded to use modern internal quota tracking (instead of legacy external files):

```
sudo umount /mnt/NAS_Storage
sudo tune2fs -O quota /dev/md0
sudo mount /mnt/NAS_Storage
```

47. Hard limits were applied to each user:

```
sudo quotatool -u alice -bq 5000G -l 5000G /mnt/NAS_Storage
sudo quotatool -u bob -bq 3000G -l 3000G /mnt/NAS_Storage
```

48. Quota configuration was verified with:

```
sudo repquota -us /mnt/NAS_Storage
```

The report confirmed Alice was assigned a 5TB hard limit and Bob was assigned a 3TB hard limit, both starting at 0 bytes used.

5.6 Samba (SMB) Network Sharing

To make the private folders accessible over the local network from Windows PCs, the Samba configuration file was edited:

```
sudo nano /etc/samba/smb.conf
```

Two share blocks were added at the bottom of the file. Each share is restricted to a single user and has appropriate file creation masks:

[Alice_Vault] — path: /mnt/NAS_Storage/alice — valid users: alice — read only: no — browseable: yes — create mask: 0700

[Bob_Vault] — path: /mnt/NAS_Storage/bob — valid users: bob — read only: no — browseable: yes — create mask: 0700

Samba was restarted to apply the new configuration:

```
sudo systemctl restart smbd
```

5.7 Map Network Drives on Windows

To provide a seamless experience, both users were asked to map their personal vault as a network drive in Windows File Explorer:

49. Open File Explorer and click This PC on the left sidebar.
50. Click the three dots at the top and select Map network drive.
51. Choose a drive letter (e.g., Z:).
52. In the Folder field, enter the path to the personal vault (for example: \\pinas.local\Alice_Vault).
53. Check Reconnect at sign-in and click Finish.

Each user now sees their personal storage vault as a drive letter in Windows, behaving just like a large USB drive.

6. Nextcloud Setup (OneDrive-Style Sync)

6.1 Architecture Decision

A comparison was made between Nextcloud and Syncthing as OneDrive-style sync solutions. Nextcloud was selected because it provides a full-featured web dashboard, mobile apps for automatic photo backup, and the ability to manage multiple user accounts. While it is heavier on the Pi's resources than Syncthing, the Raspberry Pi 5 with 8GB of RAM handles it well.

A split-brain architecture was used: Nextcloud's core application, database, and thumbnail cache are stored on the 1TB web server drive, while the actual user files remain on the 8TB RAID array in the existing quota-locked folders.

6.2 Phase 1 — Bridge the Permissions

The user folders needed to be made accessible to the Nextcloud web server process (which runs as internal user ID 33), while keeping the users isolated from each other:

```
sudo chown -R alice:33 /mnt/NAS_Storage/alice
sudo chmod -R 770 /mnt/NAS_Storage/alice
sudo chown -R bob:33 /mnt/NAS_Storage/bob
sudo chmod -R 770 /mnt/NAS_Storage/bob
```

Note: This gives both the folder owner and the Nextcloud web server full access, while everyone else has zero access. Storage quotas remain fully intact.

6.3 Phase 2 — Prepare the Web Server Drive

A dedicated directory was created on the 1TB drive for Nextcloud's brain:

```
sudo mkdir -p /mnt/Web_Server/Nextcloud_Brain
```

6.4 Phase 3 — CasaOS Installation

Nextcloud was installed through the CasaOS App Store using the custom installation option. Before clicking Install, the storage paths were manually changed to point to /mnt/Web_Server/Nextcloud_Brain instead of the default AppData location on the SD card. An additional volume mapping was added so Nextcloud can see the RAID array:

- Host: /mnt/NAS_Storage — Container: /NAS_Storage

This was applied to both the nextCloud tab and the cron tab in the container volumes settings.

6.5 Phase 4 — Initial Setup and Trusted Domain Fix

On first load, Nextcloud showed an Untrusted Domain error. This was fixed by editing the config file:

```
sudo nano /mnt/Web_Server/Nextcloud_Brain/config/config.php
```

The `trusted_domains` array was updated to include the Pi's local IP address (192.168.1.100) and local hostname (`pinas.local`):

```
'trusted_domains' => array ( 0 => 'localhost', 1 => '192.168.1.100', 2 =>
'pinas.local' )
```

6.6 Phase 5 — Connect User Vaults

The External Storage Support app was enabled in the Nextcloud Apps section. Then, under Administration Settings, each user's physical folder was linked as an external storage mount:

- For Alice: Folder name set to `/`, Storage type set to Local, Configuration path set to `/NAS_Storage/alice`, Restricted to alice only.
- For Bob: Folder name set to `/`, Storage type set to Local, Configuration path set to `/NAS_Storage/bob`, Restricted to bob only.

Each user now sees the root of their quota-locked vault when they log into Nextcloud. Green status circles confirmed both mounts were working.

6.7 Mount Options Configuration

The following mount options were configured for each vault:

- Check filesystem changes: Set to Once every direct access, so files added via the Windows mapped drive also appear in Nextcloud without delay.
- Read only: Turned OFF so users can upload and edit files.
- Enable previews: Turned ON so photo and file thumbnails appear in the web interface.
- Enable sharing: Turned ON so users can create public sharing links for individual files.
- Mac NFD encoding: Turned OFF as it is unnecessary and slows down the server.

7. Internet Access via Cloudflare Tunnel

7.1 Solution Comparison

Four options were considered for enabling internet access to the NAS:

Method	Cost	Security	User Experience
Tailscale VPN	Free	High WireGuard-based	Requires an app on each device
OpenVPN	Free	High	Complex to configure
Cloudflare Zero Trust Tunnel	Free + ~\$10/yr domain	Elite home IP hidden	Web browser, no port forwarding
Dynamic DNS + Reverse Proxy	Free	Medium	Port forwarding required

Cloudflare Zero Trust Tunnel was selected because it hides the home IP address, keeps all router ports closed, and gives users a standard HTTPS web address to connect with. The tunnel service itself is free; only a domain name is required (~\$10 per year).

7.2 Domain Name Purchase

A domain name was purchased from Name.com. The domain serves as the public address for the Nextcloud service (for example: cloud.mynas.dev).

7.3 Cloudflare Setup

Phase 1 — Cloudflare Handshake

54. A free account was created at dash.cloudflare.com, and the purchased domain was added.
55. The Free plan was selected.
56. Cloudflare's two nameservers were copied (for example: dana.ns.cloudflare.com and eric.ns.cloudflare.com).

Phase 2 — Update Nameservers at Registrar

57. Logged into Name.com and opened the domain settings.
58. The default nameservers were replaced with the two Cloudflare nameservers.
59. Changes were saved, and propagation was confirmed in the Cloudflare dashboard (typically takes 5 to 15 minutes).

Phase 3 — Create the Zero Trust Tunnel

60. In the Cloudflare dashboard, navigate to Zero Trust > Networks > Tunnels.
61. Created a new tunnel named Pi-NAS using the Cloudflared connector type.
62. Copied the Docker installation command (containing the tunnel token).

7.4 Install the Tunnel on the Pi

The tunnel was installed as a persistent Docker container so it runs in the background and restarts automatically after reboots:

```
sudo docker run -d --restart unless-stopped cloudflare/cloudflared:latest
tunnel --no-autoupdate run --token <TOKEN>
```

The tunnel status changed to Healthy in the Cloudflare dashboard within a few seconds.

7.5 Create the Public Hostname

In the Cloudflare Zero Trust dashboard, a public hostname was added to the tunnel:

- Subdomain: cloud
- Domain: mynas.dev (the purchased domain)
- Service Type: HTTP
- URL: 192.168.1.100:7580 (the local IP and port of the Nextcloud container)

7.6 Nextcloud Configuration for HTTPS

Nextcloud's config.php was updated to trust the new public domain and tell the desktop app that HTTPS is being used:

```
'trusted_domains' => array (
    0 => 'localhost',
    1 => '192.168.1.100',
    2 => 'pinas.local',
    3 => 'cloud.mynas.dev'
),
'overwriteprotocol' => 'https',
'overwritehost' => 'cloud.mynas.dev',
'overwrite.cli.url' => 'https://cloud.mynas.dev',
```

The trusted proxies array was also updated to allow the Cloudflare Docker container's internal IP to forward requests:

```
'trusted_proxies' => array (
    0 => '127.0.0.1',
    1 => '172.16.0.0/12',
    2 => '192.168.0.0/16'
)
```

The Nextcloud container was restarted via CasaOS to apply the changes. The desktop app successfully connected without showing the security warning.

7.7 CasaOS Environment Variables Fix

It was discovered that Docker's automatic startup script was occasionally overwriting the config.php settings on container restart. The permanent fix was to set the overwrite values directly as environment variables in the CasaOS container settings:

- OVERWRITEPROTOCOL = https
- OVERWRITEHOST = cloud.mynas.dev
- OVERWRITECLIURL = https://cloud.mynas.dev

These values are now applied automatically every time the container boots.

8. Client Setup

8.1 Web Browser

63. Open any web browser.
64. Navigate to <https://cloud.mynas.dev>
65. Log in with the credentials provided by the admin.
66. Files can be uploaded, downloaded, shared, and organised directly from the browser.

8.2 Mobile App

67. Download the official Nextcloud app from the App Store (iOS) or Google Play (Android).
68. Tap Add account and enter the server address: <https://cloud.mynas.dev>

Note: *The server address can also be found in the Nextcloud web dashboard under Profile > Settings > Mobile and Desktop.*

69. For secure login, use an App Password instead of the main account password:

- Go to Profile > Settings > Security > Devices and Sessions.
- Create a new app password and use it in the mobile app.

8.3 Desktop App (Windows and Mac)

70. Download the Nextcloud desktop client from <https://nextcloud.com/install/#desktop-files>
71. Install and open the app.
72. Enter the server address: <https://cloud.mynas.dev>
73. Log in using an App Password (same process as the mobile app).
74. When prompted to select a sync method, choose Use virtual files instead of downloading content immediately.

Note: *Virtual files are strongly recommended. It allows users to see all their NAS files in Windows Explorer without downloading them all at once. Files only download to the local PC when they are opened. This prevents the NAS quota from filling up the laptop's local hard drive.*

75. Click Connect. The Nextcloud folder will appear in File Explorer and behave like OneDrive.

9. Project Summary

9.1 What Was Built

This project resulted in a fully functional, self-hosted home NAS server with the following capabilities:

- Approximately 8TB of protected RAID 5 storage using three enterprise SAS drives, with fault tolerance for one drive failure.
- A 1TB dedicated storage drive for the web server and application data.
- Hardware health monitoring via GPIO-connected LEDs and a temperature buzzer.
- Isolated user storage vaults with hard disk quotas enforced at the Linux kernel level.
- Local network file sharing via Samba (Windows mapped network drives).
- A full Nextcloud cloud platform providing a OneDrive-style experience via web, mobile, and desktop apps.
- Secure internet access via a Cloudflare Zero Trust tunnel with no open ports on the home router.

9.2 Final System Architecture

Layer	Component	Purpose
Hardware	Raspberry Pi 5 + Penta SATA Hat	A computer and a SATA controller
Storage	3x 4TB SAS in RAID 5	8TB protected NAS storage
Storage	1x 1TB SATA	Web server and app data
OS	Ubuntu Server 24.04 LTS	Base operating system
Dashboard	CasaOS	Web-based management interface
Cloud Sync	Nextcloud (Docker)	OneDrive-style file sync and sharing
Local Network	Samba (SMB)	Windows mapped drive access
Internet Access	Cloudflare Zero Trust Tunnel	Secure HTTPS access from anywhere
User Isolation	Linux ACLs + Disk Quotas	Per-user storage limits and privacy
Monitoring	Python GPIO Script + systemd	LED activity and temperature alerts

9.3 Notes for Ongoing Maintenance

- If the RAID array appears as inactive after a reboot (due to drive spin-up delay), run:
`sudo mdadm --assemble --force /dev/md0 /dev/sda /dev/sdb /dev/sdc`, followed by `sudo mdadm --readwrite /dev/md0` and `sudo mount -a`.
- To monitor RAID health at any time:
`cat /proc/mdstat` and `sudo mdadm --detail /dev/md0`.
- To check disk usage per user:
`sudo repquota -us /mnt/NAS_Storage`.

- To view the health monitor log in real time:
`tail -f /usr/local/bin/health/nas_status.log.`
- Nextcloud and CasaOS should be updated regularly via the CasaOS dashboard to stay current with security patches.
- The Cloudflare tunnel runs as a Docker container with `--restart unless-stopped` and will automatically reconnect after a Pi reboot.

End of Project Report

Raspberry Pi Personal Home NAS — Version 1.0 — July 2025